

1. Práctica 2: Creación de un *Board Support Package* (BSP)

1.1. Objetivos

- Desarrollar un *Board Support Package* minimalista.
- Entender la importancia de crear, en la medida de lo posible, código no dependiente del hardware.

1.2. Fundamentos teóricos

Un *Board Support Package* o BSP comprende una colección de bibliotecas que ayudan al programador a trabajar con una placa o procesador determinado. Generalmente, incluyen definiciones, funciones, macros y demás utilidades para que el programador no tenga que trabajar directamente con código de muy bajo nivel como, por ejemplo, hemos hecho en la práctica anterior. Por ejemplo:

```
1 // En lugar de un código así
2 * ((uint32_t *) (0x40020000)) &= ~(0b11 << (7*2));
3 * ((uint32_t *) (0x40020000)) |= 0b01 << (7*2);
4
5 // Buscamos algo así
6 GPIO_SET_MODE(GPIOA, PIN7, MODE_OUTPUT);
```

Por supuesto, alguien tiene que ser el encargado de crear, por primera vez, este BSP. En sistemas empuotrados, normalmente el BSP tiene que ser creado a medida, pues la combinación de procesador, periféricos y pines es particular para cada placa desarrollada. Para la placa **32F723EDISCOVERY**, ya existe un BSP, y se anima al alumno a consultarlo y ver cómo está construido. A lo largo de estas prácticas, desarrollaremos otro, mucho más minimalista y comedido, que aporte la funcionalidad justa que necesitamos. En esta práctica comenzamos con ese trabajo.

Vamos a ver el ejemplo concreto de cómo desarrollar la función anterior, para entender mejor las partes que puede tener un BSP.

1.2.1. Definiciones de direcciones

En primer lugar, necesitamos definir las direcciones de memoria donde se encuentran todos nuestros periféricos. Esto cumple una doble función: Por un lado, si cambiaran, las podemos modificar fácilmente al estar definidas como macros. Por otro lado, hacemos el código reutilizable para procesadores de la misma familia (por ejemplo, todos los Cortex-M7 de ARM (como el **STM32F723IE** que utilizamos) reutilizan partes de los mismos BSPs).

```
1 #define GPIO_BASE_ADDR 0x40020000
2 #define GPIOA_BASE_ADDR (GPIO_BASE_ADDR + 0x0000)
3 #define GPIOB_BASE_ADDR (GPIO_BASE_ADDR + 0x0400)
4 ...
```

Table 24. GPIO register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	GPIOA_MODER	MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]		MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
	Reset value	1	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x00	GPIOB_MODER	MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]		MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0
0x00	GPIOx_MODER (where x = C..K)	MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]		MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x04	GPIOx_OTYPER (where x = A to K)	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	OT15	OT14	OT13	OT12	OT11	OT10	OT9	OT8	OT7	OT6	OT5	OT4	OT3	OT2	OT1	OT0
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figura 1: Extracto parcial de registros del controlador de GPIO

1.2.2. Definiciones de registros

Lo siguiente es definir los registros que se encuentran en dichas direcciones. Como suelen ser varios, y consecutivos, se suele definir una estructura de tipo (en lenguaje C) que los englobe todos, por facilidad de uso. Por ejemplo, mirando [rm0431 6.4.11](#), podemos ver todos los registros contenidos en cada controlador de GPIO. Se muestra la primera parte de la tabla de registros en la Figura 1

Un detalle interesante a ver aquí es que, por ejemplo, para el registro `MODER`, nos dice que tiene tres valores por defecto (reset) diferentes, según si se trata del puerto A, B, u otros. El registro `OTYPER`, sin embargo, tiene el mismo valor por defecto para todos los puertos (A-K). En cada registro, además, nos indica qué bit controla qué cosa. Para los registros `MODER`, los bits, de dos en dos, controlan el modo de GPIO (especificados en [rm0431 6.4.1](#)). Para `OTYPER`, controla el tipo de salida ([rm0431 6.4.2](#)). Si hacemos el código para tener una estructura que nos permita controlar estos registros, tendríamos algo de este estilo:

```

1 typedef struct
2 {
3     volatile uint32_t MODER;    /*!< GPIO port mode register, Address offset: 0
4     volatile uint32_t OTYPER;   /*!< GPIO port output type register, Address
5     ...                          offset: 0x04 */
6 } GPIO_TypeDef;

```

Importante observar la declaración de las variables como tipo `volatile`, ya que, además de modificarse internamente desde el procesador, son registros cuyo valor puede modificarse desde fuera (por el propio control del periférico). Con la palabra clave `volatile` conseguimos que el compilador no optimice esas variables y siempre las consulte directamente de memoria.

Pregunta: ¿A qué se debe que todos los registros queden colocados en direcciones sucesivas de memoria al ser declarados con esta sintaxis?

Pregunta: Explora qué hace exactamente el compilador al trabajar con objetos volátiles.

1.2.3. Gestión de registros

Tenemos definidas las direcciones, y las estructuras de registros. Ahora necesitamos tener una forma de escribir en dichas estructuras. Para ello, vamos a crear punteros a las direcciones de memoria base, con el tipo de estructura definido:

```
1 #define GPIOA ((GPIO_TypeDef *) GPIOA_BASE_ADDR)
```

Si la base del controlador se encuentra en la dirección `0x40020000` (como es el caso de GPIOA), los registros se encontrarán a continuación (el primero `MODER` en `0x40020000`, el segundo `OTYPER` en `0x40020004`, y así sucesivamente). Así, lo que conseguimos es que, posteriormente, en el código de nuestro programa, podamos hacer algo de este estilo:

```
1 GPIOA->MODER = 0xA8000000;  
2 uint32_t temp = GPIOA->MODER;
```

Exactamente equivalente a haber escrito (o leído) desde las direcciones de memoria correspondientes. Es decir, de una forma legible, acceder (para escribir o leer) los contenidos de estos registros, que a la vez sirven para control de los periféricos.

Pregunta: Cada registro normalmente controla, desde cada uno de sus bits, elementos diferentes del periférico. ¿Cómo puedo leer (o escribir) en unos bits concretos del registro?

1.2.4. Funciones de control de registros

Aunque ahora es más cómodo trabajar con los registros, lo ideal es contar con funciones que nos ayuden, aún más, a abstraer todos estos códigos de bajo nivel que son propensos a errores por parte del programador. Por ejemplo, ahora mismo, si quisiéramos cambiar el modo del pin 7, del puerto A, a *output*, deberíamos hacer algo de este estilo:

```
1 GPIOA->MODER &= ~(0b11 << (7*2));  
2 GPIOA->MODER |= 0b01 << (7*2);
```

Es decir, borramos primero los dos bits de las posiciones 14 y 15 (Ver tabla anterior) y luego les ponemos el valor "01" para configurarlo a nuestro gusto. Pero, ¿y si ahora queremos cambiar el pin 4 del puerto B? ¿o ponerlo en modo *input*?. Es mejor crear una función genérica que nos permita hacer todo.

Aunque esto ya es a gusto del consumidor, la sugerencia para estas funciones es hacerlas lo más genéricas posibles. En este caso, deberíamos poder permitir trabajar con cualquier puerto, cualquier pin, y cualquier modo. Por ejemplo, podemos hacer algo así:

5.3.10 RCC AHB1 peripheral clock register (RCC_AHB1ENR)

Address offset: 0x30

Reset value: 0x0010 0000

Access: no wait state, word, half-word and byte access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	OTGHS ULPIEN	OTGHS EN	Res.	Res.	Res.	Res.	Res.	Res.	DMA2 EN	DMA1 EN	DTCMRA MEN	Res.	BKPSR AMEN	Res.	Res.
	rw	rw							rw	rw	rw		rw		
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	CRC EN	Res.	Res.	Res.	GPIOI EN	GPIOH EN	GPIOG EN	GPIOF EN	GPIOE EN	GPIOD EN	GPIOC EN	GPIOB EN	GPIOA EN
			rw				rw	rw	rw	rw	rw	rw	rw	rw	rw

Figura 2: Detalle de bits del registro **AHB1ENR**

```
1 typedef enum {
2     GPIO_MODE_INPUT    = 0x00,
3     GPIO_MODE_OUTPUT   = 0x01,
4     GPIO_MODE_ALT      = 0x02,
5     GPIO_MODE_ANALOG   = 0x03
6 } GPIO_Mode_t;
7
8 static inline void GPIO_MODE_SET(volatile GPIO_TypeDef *GPIOx, uint32_t pin,
9     GPIO_Mode_t mode) {
10     uint32_t temp = GPIOx->MODER; //leemos registro
11     temp &= ~(0b11 << (pin * 2)); //borramos modo actual
12     temp |= (mode << (pin * 2)); //ponemos modo nuevo
13     GPIOx->MODER = temp; //escribimos de vuelta
14 }
```

Con esto, ya podríamos hacer desde nuestro código principal, lo siguiente:

```
1 GPIO_MODE_SET(GPIOA, 7, GPIO_MODE_OUTPUT);
```

1.2.5. Gestión directa bit a bit de registros

Otros registros, como por ejemplo **AHB1ENR** ([rm0431 5.3.10](#)) tienen una estructura que no es regular, ya que cada bit controla una cosa diferente, como podemos ver en la Figura 2.

Para estos casos, C nos permite una sintaxis como la siguiente, que nos deja acceder individualmente a nivel de bit a los registros:

```
1 //Base address
2 #define RCC_AHB1EN_BASE_ADDR 0x40023830
3
4 //Structure for Reset and Clock Control peripheral clock register 1
5 typedef union {
6     volatile uint32_t reg;
7
8     // 2. Access individual bits or bit-groups for the register above
9     struct {
10         volatile uint32_t GPIOAEN : 1; // Bit 0: Enable peripheral (1 bit
11     )
12 }
```

```

11         volatile uint32_t GPIOBEN      : 1;  // Bit 1: Enable peripheral (1 bit
12         )
13     ...
14 } bits;
15 } RCC_AHB_ENR_TypeDef;
16
17 //Handle:
18 #define RCC_AHB1ENR          ((RCC_AHB_ENR_TypeDef *) RCC_AHB1EN_BASE_ADDR)

```

Posteriormente, podemos acceder a los campos individuales de la siguiente manera:

```

1 RCC_AHB1ENR->bits.GPIOAEN = 1;

```

Este registro en concreto es muy importante, pues activa los periféricos. De no activarlos, no funcionarán aunque se configuren adecuadamente.

1.3. Desarrollo de la práctica

Crea un BSP para que, en el código de la práctica anterior, no haya que acceder directamente a registros del periférico GPIOA. Necesitarás funciones para configurar `MODER`, `OTYPER`, `OSPEEDR`, `PUPDR` y también para escribir 1 o 0 (esto lo puedes hacer desde el registro `ODR` o `BSRR`). Consulta [rm0431 6.4](#) para ver todos los registros y sus funcionalidades. Deberás:

1. Completar la definición de tipo para incluir todos los registros. También completar la definición de tipo de `AHB1ENR`.
2. Crear las funciones, junto con sus definiciones de valores, para controlar los diferentes registros.
3. Empaquetar toda esta funcionalidad en un fichero `bsp.h` que irá dentro de la carpeta `Inc` del proyecto.
4. Adaptar el código original `main.c` de la práctica para utilizar el BSP.

1.4. Para hacer más

Abre el proyecto BSP de ejemplo (dentro del paquete `STM32CubeF7`) e investiga cómo están organizadas las funciones, definiciones, registros... Para navegar por el proyecto, puedes ir haciendo `ctrl+click` en las llamadas a funciones o variables, y te llevará al fichero donde están definidas.